

Bulk_quote 的对象当成 Quote 的对象来使用。

派生类必须在其内部对所有重新定义的虚函数进行声明。派生类可以在这样的函数之前加上 virtual 关键字，但是并不是非得这么做。出于 15.3 节（第 538 页）将要解释的原因，C++11 新标准允许派生类显式地注明它将使用哪个成员函数改写基类的虚函数，具体措施是在该函数的形参列表之后增加一个 override 关键字。

动态绑定

通过使用动态绑定（dynamic binding），我们能用同一段代码分别处理 Quote 和 Bulk_quote 的对象。例如，当要购买的书籍和购买的数量都已知时，下面的函数负责打印总的费用：

```
// 计算并打印销售给定数量的某种书籍所得的费用
double print_total(ostream &os,
                  const Quote &item, size_t n)
{
    // 根据传入 item 形参的对象类型调用 Quote::net_price
    // 或者 Bulk_quote::net_price
    double ret = item.net_price(n);
    os << "ISBN: " << item.isbn() // 调用 Quote::isbn
        << " # sold: " << n << " total due: " << ret << endl;
    return ret;
}
```

该函数非常简单：它返回调用 net_price() 的结果，并将该结果连同调用 isbn() 的结果一起打印出来。

关于上面的函数有两个有意思的结论：因为函数 print_total 的 item 形参是基类 Quote 的一个引用，所以出于 15.2.3 节（第 534 页）将要解释的原因，我们既能使用基类 Quote 的对象调用该函数，也能使用派生类 Bulk_quote 的对象调用它；又因为 print_total 是使用引用类型调用 net_price 函数的，所以出于 15.2.1 节（第 528 页）将要解释的原因，实际传入 print_total 的对象类型将决定到底执行 net_price 的哪个版本：

```
// basic 的类型是 Quote；bulk 的类型是 Bulk_quote
print_total(cout, basic, 20); // 调用 Quote 的 net_price
print_total(cout, bulk, 20); // 调用 Bulk_quote 的 net_price
```

第一条调用句将 Quote 对象传入 print_total，因此当 print_total 调用 net_price 时，执行的是 Quote 的版本；在第二条调用语句中，实参的类型是 Bulk_quote，因此执行的是 Bulk_quote 的版本（计算打折信息）。因为在上述过程中函数的运行版本由实参决定，即在运行时选择函数的版本，所以动态绑定有时又被称为运行时绑定（run-time binding）。

594



在 C++ 语言中，当我们使用基类的引用（或指针）调用一个虚函数时将发生动态绑定。

15.2 定义基类和派生类

定义基类和派生类的方式在很多方面都与我们已知的定义其他类的方式类似，但是也有一些不同之处。本节将介绍在定义有继承关系的类时可能用到的基本特性。



15.2.1 定义基类

我们首先完成 Quote 类的定义：

```
class Quote {
public:
    Quote() = default;           // 关于=default 请参见 7.1.4 节 (第 237 页)
    Quote(const std::string &book, double sales_price):
        bookNo(book), price(sales_price) { }
    std::string isbn() const { return bookNo; }
    // 返回给定数量的书籍的销售总额
    // 派生类负责改写并使用不同的折扣计算算法
    virtual double net_price(std::size_t n) const
        { return n * price; }
    virtual ~Quote() = default;  // 对析构函数进行动态绑定
private:
    std::string bookNo;         // 书籍的 ISBN 编号
protected:
    double price = 0.0;         // 代表普通状态下不打折的价格
};
```

对于上面这个类来说,新增的部分是在 `net_price` 函数和析构函数之前增加的 `virtual` 关键字以及最后的 `protected` 访问说明符。我们将在 15.7.1 节 (第 552 页) 详细介绍虚析构函数的知识,现在只需记住作为继承关系中根节点类通常都会定义一个虚析构函数。



基类通常都应该定义一个虚析构函数,即使该函数不执行任何实际操作也是如此。

成员函数与继承

派生类可以继承其基类的成员,然而当遇到如 `net_price` 这样与类型相关的操作时,派生类必须对其重新定义。换句话说,派生类需要对这些操作提供自己的新定义以覆盖 (override) 从基类继承而来的旧定义。

在 C++ 语言中,基类必须将它的两种成员函数区分开来:一种是基类希望其派生类进行覆盖的函数;另一种是基类希望派生类直接继承而不要改变的函数。对于前者,基类通常将其定义为虚函数 (virtual)。当我们使用指针或引用调用虚函数时,该调用将被动态绑定。根据引用或指针所绑定的对象类型不同,该调用可能执行基类的版本,也可能执行某个派生类的版本。

基类通过在其成员函数的声明语句之前加上关键字 `virtual` 使得该函数执行动态绑定。任何构造函数之外的非静态函数 (参见 7.6 节,第 268 页) 都可以是虚函数。关键字 `virtual` 只能出现在类内部的声明语句之前而不能用于类外部的函数定义。如果基类把一个函数声明成虚函数,则该函数在派生类中隐式地也是虚函数。我们将在 15.3 节 (第 536 页) 介绍更多关于虚函数的知识。

成员函数如果没被声明为虚函数,则其解析过程发生在编译时而非运行时。对于 `isbn` 成员来说这正是我们希望看到的结果。`isbn` 函数的执行与派生类的细节无关,不管作用于 `Quote` 对象还是 `Bulk_quote` 对象, `isbn` 函数的行为都一样。在我们的继承层次关系中只有一个 `isbn` 函数,因此也就不存在调用 `isbn()` 时到底执行哪个版本的疑问。

访问控制与继承

派生类可以继承定义在基类中的成员，但是派生类的成员函数不一定有权访问从基类继承而来的成员。和其他使用基类的代码一样，派生类能访问公有成员，而不能访问私有成员。不过在某些时候基类中还有这样一种成员，基类希望它的派生类有权访问该成员，同时禁止其他用户访问。我们用受保护的（protected）访问运算符说明这样的成员。

我们的 Quote 类希望它的派生类定义各自的 net_price 函数，因此派生类需要访问 Quote 的 price 成员。此时我们将 price 定义成受保护的。与之相反，派生类访问 bookNo 成员的方式与其他用户是一样的，都是通过调用 isbn 函数，因此 bookNo 被定义成私有的，即使是 Quote 派生出来的类也不能直接访问它。我们将在 15.5 节（第 542 页）介绍更多关于受保护成员的知识。

15.2.1 节练习

练习 15.1：什么是虚成员？

练习 15.2：protected 访问说明符与 private 有何区别？

练习 15.3：定义你自己的 Quote 类和 print_total 函数。

15.2.2 定义派生类

596

派生类必须通过使用类派生列表（class derivation list）明确指出它是从哪个（哪些）基类继承而来的。类派生列表的形式是：首先是一个冒号，后面紧跟以逗号分隔的基类列表，其中每个基类前面可以有以下三种访问说明符中的一个：public、protected 或者 private。

派生类必须将其继承而来的成员函数中需要覆盖的那些重新声明，因此，我们的 Bulk_quote 类必须包含一个 net_price 成员：

```
class Bulk_quote : public Quote {           // Bulk_quote 继承自 Quote
public:
    Bulk_quote() = default;
    Bulk_quote(const std::string&, double, std::size_t, double);
    // 覆盖基类的函数版本以实现基于大量购买的折扣政策
    double net_price(std::size_t) const override;
private:
    std::size_t min_qty = 0;                // 适用折扣政策的最低购买量
    double discount = 0.0;                 // 以小数表示的折扣额
};
```

我们的 Bulk_quote 类从它的基类 Quote 那里继承了 isbn 函数和 bookNo、price 等数据成员。此外，它还定义了 net_price 的新版本，同时拥有两个新增加的数据成员 min_qty 和 discount。这两个成员分别用于说明享受折扣所需购买的最低数量以及一旦该数量达到之后具体的折扣信息。

我们将在 15.5 节（第 543 页）详细介绍派生列表中用到的访问说明符。现在，我们只需知道访问说明符的作用是控制派生类从基类继承而来的成员是否对派生类的用户可见。

如果一个派生是公有的，则基类的公有成员也是派生类接口的组成部分。此外，我们
能将公有派生类型的对象绑定到基类的引用或指针上。因为我们在派生列表中使用了

public，所以 Bulk_quote 的接口隐式地包含 isbn 函数，同时在任何需要 Quote 的引用或指针的地方我们都能使用 Bulk_quote 的对象。

大多数类都只继承自一个类，这种形式的继承被称作“单继承”，它构成了本章的主题。关于派生列表中含有多于一个基类的情况将在 18.3 节（第 710 页）中介绍。

派生类中的虚函数

派生类经常（但不总是）覆盖它继承的虚函数。如果派生类没有覆盖其基类中的某个虚函数，则该虚函数的行为类似于其他的普通成员，派生类会直接继承其在基类中的版本。

C++11

派生类可以在它覆盖的函数前使用 virtual 关键字，但不是非得这么做。我们将在 15.3 节（第 538 页）介绍其原因，C++11 新标准允许派生类显式地注明它使用某个成员函数覆盖了它继承的虚函数。具体做法是在形参列表后面、或者在 const 成员函数（参见 7.1.2 节，第 231 页）的 const 关键字后面、或者在引用成员函数（参见 13.6.3 节，第 483 页）的引用限定符后面添加一个关键字 override。

597 ➤ 派生类对象及派生类向基类的类型转换

一个派生类对象包含多个组成部分：一个含有派生类自己定义的（非静态）成员的子对象，以及一个与该派生类继承的基类对应的子对象，如果有多个基类，那么这样的子对象也有多个。因此，一个 Bulk_quote 对象将包含四个数据元素：它从 Quote 继承而来的 bookNo 和 price 数据成员，以及 Bulk_quote 自己定义的 min_qty 和 discount 成员。

C++ 标准并没有明确规定派生类的对象在内存中如何分布，但是我们可以认为 Bulk_quote 的对象包含如图 15.1 所示的两部分。



在一个对象中，继承自基类的部分和派生类自定义的部分不一定是连续存储的。图 15.1 只是表示类工作机制的概念模型，而非物理模型。

图 15.1: Bulk_quote 对象的概念结构

因为在派生类对象中含有与其基类对应的组成部分，所以我们可以把派生类的对象当成基类对象来使用，而且我们也能将基类的指针或引用绑定到派生类对象中的基类部分上。

```
Quote item;           // 基类对象
Bulk_quote bulk;       // 派生类对象
Quote *p = &item;      // p 指向 Quote 对象
p = &bulk;             // p 指向 bulk 的 Quote 部分
Quote &r = bulk;       // r 绑定到 bulk 的 Quote 部分
```

这种转换通常称为派生类到基类的（derived-to-base）类型转换。和其他类型转换一样，编译器会隐式地执行派生类到基类的转换（参见 4.11 节，第 141 页）。

这种隐式特性意味着我们可以把派生类对象或者派生类对象的引用用在需要基类引

用的地方；同样的，我们也可以把派生类对象的指针用在需要基类指针的地方。

Note

在派生类对象中含有与其基类对应的组成部分，这一事实是继承的关键所在。

派生类构造函数

598

尽管在派生类对象中含有从基类继承而来的成员，但是派生类并不能直接初始化这些成员。和其他创建了基类对象的代码一样，派生类也必须使用基类的构造函数来初始化它的基类部分。

Note

每个类控制它自己的成员初始化过程。

派生类对象的基类部分与派生类对象自己的数据成员都是在构造函数的初始化阶段（参见 7.5.1 节，第 258 页）执行初始化操作的。类似于我们初始化成员的过程，派生类构造函数同样是通过构造函数初始化列表来将实参传递给基类构造函数的。例如，接受四个参数的 Bulk_quote 构造函数如下所示：

```
Bulk_quote(const std::string& book, double p,
            std::size_t qty, double disc) :
    Quote(book, p), min_qty(qty), discount(disc) { }
    // 与之前一致
};
```

该函数将它的前两个参数（分别表示 ISBN 和价格）传递给 Quote 的构造函数，由 Quote 的构造函数负责初始化 Bulk_quote 的基类部分（即 bookNo 成员和 price 成员）。当（空的）Quote 构造函数体结束后，我们构建的对象的基类部分也就完成初始化了。接下来初始化由派生类直接定义的 min_qty 成员和 discount 成员。最后运行 Bulk_quote 构造函数的（空的）函数体。

除非我们特别指出，否则派生类对象的基类部分会像数据成员一样执行默认初始化。如果想使用其他的基类构造函数，我们需要以类名加圆括号内的实参列表的形式为构造函数提供初始值。这些实参将帮助编译器决定到底应该选用哪个构造函数来初始化派生类对象的基类部分。

Note

首先初始化基类的部分，然后按照声明的顺序依次初始化派生类的成员。

派生类使用基类的成员

派生类可以访问基类的公有成员和受保护成员：

```
// 如果达到了购买书籍的某个最低限量值，就可以享受折扣价格了
double Bulk_quote::net_price(size_t cnt) const
{
    if (cnt >= min_qty)
        return cnt * (1 - discount) * price;
    else
        return cnt * price;
}
```

该函数产生一个打折后的价格：如果给定的数量超过了 min_qty，则将 discount（一

599

个小于 1 大于 0 的数) 作用于 price。

我们将在 15.6 节 (第 547 页) 进一步讨论作用域, 目前只需要了解派生类的作用域嵌套在基类的作用域之内。因此, 对于派生类的一个成员来说, 它使用派生类成员 (例如 min_qty 和 discount) 的方式与使用基类成员 (例如 price) 的方式没什么不同。

关键概念: 遵循基类的接口

必须明确一点: 每个类负责定义各自的接口。要想与类的对象交互必须使用该类的接口, 即使这个对象是派生类的基类部分也是如此。

因此, 派生类对象不能直接初始化基类的成员。尽管从语法上来说我们可以在派生类构造函数体内给它的公有或受保护的基类成员赋值, 但是最好不要这么做。和使用基类的其他场合一样, 派生类应该遵循基类的接口, 并且通过调用基类的构造函数来初始化那些从基类中继承而来的成员。

继承与静态成员

如果基类定义了一个静态成员 (参见 7.6 节, 第 268 页), 则在整个继承体系中只存在该成员的唯一定义。不论从基类中派生出来多少个派生类, 对于每个静态成员来说都只存在唯一的实例。

```
class Base {
public:
    static void statmem();
};
class Derived : public Base {
    void f(const Derived&);
};
```

静态成员遵循通用的访问控制规则, 如果基类中的成员是 private 的, 则派生类无权访问它。假设某静态成员是可访问的, 则我们既能通过基类使用它也能通过派生类使用它:

```
void Derived::f(const Derived &derived_obj)
{
    Base::statmem();           // 正确: Base 定义了 statmem
    Derived::statmem();        // 正确: Derived 继承了 statmem
    // 正确: 派生类的对象能访问基类的静态成员
    derived_obj.statmem();      // 通过 Derived 对象访问
    statmem();                 // 通过 this 对象访问
}
```

600 派生类的声明

派生类的声明与其他类差别不大 (参见 7.3.3 节, 第 250 页), 声明中包含类名但是不包含它的派生列表:

```
class Bulk_quote : public Quote; // 错误: 派生列表不能出现在这里
class Bulk_quote;               // 正确: 声明派生类的正确方式
```

一条声明语句的目的是令程序知晓某个名字的存在以及该名字表示一个什么样的实体, 如一个类、一个函数或一个变量等。派生列表以及与定义有关的其他细节必须与类的主体一起出现。

被用作基类的类

如果我们想将某个类用作基类，则该类必须已经定义而非仅仅声明：

```
class Quote;                                // 声明但未定义
// 错误: Quote 必须被定义
class Bulk_quote : public Quote { ... };
```

这一规定的原因显而易见：派生类中包含并且可以使用它从基类继承而来的成员，为了使用这些成员，派生类当然要知道它们是什么。因此该规定还有一层隐含的意思，即一个类不能派生它本身。

一个类是基类，同时它也可以是一个派生类：

```
class Base { /* ... */ };
class D1: public Base { /* ... */ };
class D2: public D1 { /* ... */ };
```

在这个继承关系中，Base 是 D1 的**直接基类**（direct base），同时是 D2 的**间接基类**（indirect base）。直接基类出现在派生列表中，而间接基类由派生类通过其直接基类继承而来。

每个类都会继承直接基类的所有成员。对于一个最终的派生类来说，它会继承其直接基类的成员；该直接基类的成员又含有其基类的成员；依此类推直至继承链的顶端。因此，最终的派生类将包含它的直接基类的子对象以及每个间接基类的子对象。

防止继承的发生

有时我们会定义这样一种类，我们不希望其他类继承它，或者不想考虑它是否适合作为一个基类。为了实现这一目的，C++11 新标准提供了一种防止继承发生的方法，即在类名后跟一个关键字 final：

C++
11

```
class NoDerived final { /* */ };           // NoDerived 不能作为基类
class Base { /* */ };
// Last 是 final 的；我们不能继承 Last
class Last final : Base { /* */ };         // Last 不能作为基类
class Bad : NoDerived { /* */ };           // 错误: NoDerived 是 final 的
class Bad2 : Last { /* */ };               // 错误: Last 是 final 的
```

15.2.2 节练习

601

练习 15.4：下面哪条声明语句是不正确的？请解释原因。

```
class Base { ... };
(a) class Derived : public Derived { ... };
(b) class Derived : private Base { ... };
(c) class Derived : public Base;
```

练习 15.5：定义你自己的 Bulk_quote 类。

练习 15.6：将 Quote 和 Bulk_quote 的对象传给 15.2.1 节（第 529 页）练习中的 print_total 函数，检查该函数是否正确。

练习 15.7：定义一个类使其实现一种数量受限的折扣策略，具体策略是：当购买书籍的数量不超过一个给定的限量时享受折扣，如果购买量一旦超过了限量，则超出的部分将以原价销售。



15.2.3 类型转换与继承



理解基类和派生类之间的类型转换是理解 C++ 语言面向对象编程的关键所在。

通常情况下，如果我们想把引用或指针绑定到一个对象上，则引用或指针的类型应与对象的类型一致（参见 2.3.1 节，第 46 页和 2.3.2 节，第 47 页），或者对象的类型含有一个可接受的 `const` 类型转换规则（参见 4.11.2 节，第 144 页）。存在继承关系的类是一个重要的例外：我们可以将基类的指针或引用绑定到派生类对象上。例如，我们可以用 `Quote&` 指向一个 `Bulk_quote` 对象，也可以把一个 `Bulk_quote` 对象的地址赋给一个 `Quote*`。

可以将基类的指针或引用绑定到派生类对象上有一层极为重要的含义：当使用基类的引用（或指针）时，实际上我们并不清楚该引用（或指针）所绑定对象的真实类型。该对象可能是基类的对象，也可能是派生类的对象。



和内置指针一样，智能指针类（参见 12.1 节，第 400 页）也支持派生类向基类的类型转换，这意味着我们可以将一个派生类对象的指针存储在一个基类的智能指针内。



静态类型与动态类型

当我们使用存在继承关系的类型时，必须将一个变量或其他表达式的**静态类型**（static type）与该表达式表示对象的**动态类型**（dynamic type）区分开来。表达式的静态类型在编译时总是已知的，它是变量声明时的类型或表达式生成的类型；动态类型则是变量或表达式表示的内存中的对象的类型。动态类型直到运行时才可知。

602

例如，当 `print_total` 调用 `net_price` 时（参见 15.1 节，第 527 页）：

```
double ret = item.net_price(n);
```

我们知道 `item` 的静态类型是 `Quote&`，它的动态类型则依赖于 `item` 绑定的实参，动态类型直到在运行时调用该函数时才会知道。如果我们传递一个 `Bulk_quote` 对象给 `print_total`，则 `item` 的静态类型将与它的动态类型不一致。如前所述，`item` 的静态类型是 `Quote&`，而在此例中它的动态类型则是 `Bulk_quote`。

如果表达式既不是引用也不是指针，则它的动态类型永远与静态类型一致。例如，`Quote` 类型的变量永远是一个 `Quote` 对象，我们无论如何都不能改变该变量对应的对象的类型。



基类的指针或引用的静态类型可能与其动态类型不一致，读者一定要理解其中的原因。

不存在从基类向派生类的隐式类型转换……

之所以存在派生类向基类的类型转换是因为每个派生类对象都包含一个基类部分，而基类的引用或指针可以绑定到该基类部分上。一个基类的对象既可以以独立的形式存在，也可以作为派生类对象的一部分存在。如果基类对象不是派生类对象的一部分，则它只含有基类定义的成员，而不含有派生类定义的成员。

因为一个基类的对象可能是派生类对象的一部分，也可能不是，所以不存在从基类向派生类的自动类型转换：


```
Quote base;
Bulk_quote* bulkP = &base;           // 错误：不能将基类转换成派生类
Bulk_quote& bulkRef = base;          // 错误：不能将基类转换成派生类
```

如果上述赋值是合法的，则我们有可能会使用 `bulkP` 或 `bulkRef` 访问 `base` 中本不存在的成员。

除此之外还有一种情况显得有点特别，即使一个基类指针或引用绑定在一个派生类对象上，我们也不能执行从基类向派生类的转换：

```
Bulk_quote bulk;
Quote *itemP = &bulk;                // 正确：动态类型是 Bulk_quote
Bulk_quote *bulkP = itemP;           // 错误：不能将基类转换成派生类
```

编译器在编译时无法确定某个特定的转换在运行时是否安全，这是因为编译器只能通过检查指针或引用的静态类型来推断该转换是否合法。如果在基类中含有一个或多个虚函数，我们可以使用 `dynamic_cast`（参见 19.2.1 节，第 730 页）请求一个类型转换，该转换的安全检查将在运行时执行。同样，如果我们已知某个基类向派生类的转换是安全的，则我们可以使用 `static_cast`（参见 4.11.3 节，第 144 页）来强制覆盖掉编译器的检查工作。

……在对象之间不存在类型转换



派生类向基类的自动类型转换只对指针或引用类型有效，在派生类类型和基类类型之间不存在这样的转换。很多时候，我们确实希望将派生类对象转换成它的基类类型，但是这种转换的实际发生过程往往与我们期望的有所差别。

请注意，当我们初始化或赋值一个类类型的对象时，实际上是在调用某个函数。当执行初始化时，我们调用构造函数（参见 13.1.1 节，第 440 页和 13.6.2 节，第 473 页）；而当执行赋值操作时，我们调用赋值运算符（参见 13.1.2 节，第 443 页和 13.6.2 节，第 474 页）。这些成员通常都包含一个参数，该参数的类型是类类型的 `const` 版本的引用。

因为这些成员接受引用作为参数，所以派生类向基类的转换允许我们给基类的拷贝/移动操作传递一个派生类的对象。这些操作不是虚函数。当我们给基类的构造函数传递一个派生类对象时，实际运行的构造函数是基类中定义的那个，显然该构造函数只能处理基类自己的成员。类似的，如果我们将一个派生类对象赋值给一个基类对象，则实际运行的赋值运算符也是基类中定义的那个，该运算符同样只能处理基类自己的成员。

例如，我们的书店类使用了合成版本的拷贝和赋值操作（参见 13.1.1 节，第 440 页和 13.1.2 节，第 444 页）。关于拷贝控制与继承的知识将在 15.7.2 节（第 552 页）做更详细的介绍，现在我们只需要知道合成版本会像其他类一样逐成员地执行拷贝或赋值操作：

```
Bulk_quote bulk;                // 派生类对象
Quote item(bulk);               // 使用 Quote::Quote(const Quote&) 构造函数
item = bulk;                    // 调用 Quote::operator=(const Quote&)
```

当构造 `item` 时，运行 `Quote` 的拷贝构造函数。该函数只能处理 `bookNo` 和 `price` 两个成员，它负责拷贝 `bulk` 中 `Quote` 部分的成员，同时忽略掉 `bulk` 中 `Bulk_quote` 部分的成员。类似的，对于将 `bulk` 赋值给 `item` 的操作来说，只有 `bulk` 中 `Quote` 部分的成员被赋值给 `item`。

因为在上述过程中会忽略 `Bulk_quote` 部分，所以我们可以说 `bulk` 的 `Bulk_quote` 部分被切掉（sliced down）了。



当我们用一个派生类对象为一个基类对象初始化或赋值时，只有该派生类对象中的基类部分会被拷贝、移动或赋值，它的派生类部分将被忽略掉。

15.2.3 节练习

练习 15.8: 给出静态类型和动态类型的定义。

练习 15.9: 在什么情况下表达式的静态类型可能与动态类型不同？请给出三个静态类型与动态类型不同的例子。

练习 15.10: 回忆我们在 8.1 节（第 279 页）进行的讨论，解释第 284 页中将 ifstream 传递给 Sales_data 的 read 函数的程序是如何工作的。

关键概念：存在继承关系的类型之间的转换规则

要想理解在具有继承关系的类之间发生的类型转换，有三点非常重要：

- 从派生类向基类的类型转换只对指针或引用类型有效。
- 基类向派生类不存在隐式类型转换。
- 和任何其他成员一样，派生类向基类的类型转换也可能会由于访问受限而变得不可行。我们将在 15.5 节（第 544 页）详细介绍可访问性的问题。

尽管自动类型转换只对指针或引用类型有效，但是继承体系中的大多数类仍然（显式或隐式地）定义了拷贝控制成员（参见第 13 章）。因此，我们通常能够将一个派生类对象拷贝、移动或赋值给一个基类对象。不过需要注意的是，这种操作只处理派生类对象的基类部分。



15.3 虚函数

如前所述，在 C++ 语言中，当我们使用基类的引用或指针调用一个虚成员函数时会执行动态绑定（参见 15.1 节，第 527 页）。因为我们直到运行时才能知道到底调用了哪个版本的虚函数，所以所有虚函数都必须有定义。通常情况下，如果我们不使用某个函数，则无须为该函数提供定义（参见 6.1.2 节，第 186 页）。但是我们必须为每一个虚函数都提供定义，而不管它是否被用到了，这是因为连编译器也无法确定到底会使用哪个虚函数。

对虚函数的调用可能在运行时才被解析

当某个虚函数通过指针或引用调用时，编译器产生的代码直到运行时才能确定应该调用哪个版本的函数。被调用的函数是与绑定到指针或引用上的对象的动态类型相匹配的那一个。

举个例子，考虑 15.1 节（第 527 页）的 print_total 函数，该函数通过其名为 item 的参数来进一步调用 net_price，其中 item 的类型是 Quote&。因为 item 是引用而且 net_price 是虚函数，所以我们到底调用 net_price 的哪个版本完全依赖于运行时绑定到 item 的实参的实际（动态）类型：

```
Quote base("0-201-82470-1", 50);
print_total(cout, base, 10);           // 调用 Quote::net_price
Bulk_quote derived("0-201-82470-1", 50, 5, .19);
```